

■ WinniKnight: MindGuardians

Documentación de Integración Funcional — Web (Next.js)

Entrega 4 — Paso 4 | Fundamentos de Aplicaciones Web | Grupo 3

Integrante	Carné	Correo
Pablo Urbina	24001508	24001508@galileo.edu
Angel Camargo	24003664	angel.camargo@galileo.edu

1. Descripción del Proyecto

WinniKnight: MindGuardians es una plataforma web construida con **Next.js 14** que complementa la app Android del mismo proyecto. Comparte la misma base de datos Firebase con la app Android — los datos del héroe son sincronizados entre ambas plataformas en tiempo real.

La plataforma web ofrece una experiencia de dashboard completo orientada a escritorio, con componentes dedicados para cada sección: estadísticas de vitalidad, historial de batallas, inventario de equipo, tienda, ranking global, perfil personalizable, oráculo de IA y guía de usuario. La arquitectura está optimizada para que ninguna credencial sensible llegue al navegador.

2. Arquitectura del Sistema

El sistema web sigue una arquitectura de tres capas:

- **Navegador (Next.js Client)** — interfaz de usuario, Firebase Auth y Firestore SDK
- **Servidor Next.js (API Routes)** — proxy seguro hacia la Cloud Function de Python
- **Firebase Cloud Functions (Python)** — backend que protege la API key de Gemini

La URL de la Cloud Function (**ORACLE_FUNCTION_URL**) y la API key de Gemini nunca llegan al navegador. Viven en variables de entorno del servidor Next.js, fuera del bundle que se envía al cliente.

2.1 Flujo del Oráculo

Paso	Origen	Destino	Protocolo
1	GeminiOracle.tsx	POST /api/oracle	fetch() desde browser
2	route.ts (servidor)	ORACLE_FUNCTION_URL	fetch() server-side con variable env
3	Cloud Function	Gemini API	HTTPS + API key en Firebase Secrets
4	Gemini API	Cloud Function	JSON {candidates[0].content.parts[0].text}
5	Cloud Function	route.ts	JSON {reply: string}
6	route.ts	Navegador	NextResponse.json({reply})

2.2 Flujo de Datos de Firestore (Client-side)

Las lecturas de Firestore (usuarios, batallas, ranking, inventario, tienda) se realizan directamente desde el navegador usando el SDK de Firebase. Esto es seguro porque las operaciones requieren autenticación mediante Firebase Auth y las reglas de Firestore las protegen.

3. Stack Tecnológico

Capa	Tecnología
Framework	Next.js 14.2.5 (App Router)
Lenguaje	TypeScript 5.5.3
UI Framework	React 18.3.1

Estilos	Tailwind CSS 3.4.6 con tokens personalizados (mq-*)
Iconos	Lucide React 0.383.0
Gráficas	Recharts 2.12.7
Autenticación	Firebase Auth (email/password)
Base de datos	Firebase Firestore SDK 10.14.1
Backend IA	Next.js API Routes + Cloud Functions (Python compartida con Android)
IA generativa	Gemini API (gemini-2.5-flash via Cloud Function)
Rendering	CSR para dashboard, SSR implícito en API Routes
Routing	Next.js App Router con layout.tsx raíz

4. Dependencias — package.json

Librería	Versión	Uso
next	14.2.5	Framework principal (App Router, API Routes, SSR/CSR)
react	18.3.1	UI, Hooks (useState, useEffect, useCallback, useRef)
react-dom	18.3.1	Renderizado al DOM del navegador
firebase	10.14.1	Auth + Firestore SDK cliente
recharts	2.12.7	Gráficas de vitalidad (BarChart, LineChart)
lucide-react	0.383.0	Iconos vectoriales (LayoutDashboard, Sword, Sparkles, etc.)
tailwindcss	3.4.6	Utility-first CSS con tokens de diseño personalizados
typescript	5.5.3	Tipado estático completo del proyecto
postcss	8.x	Procesamiento de CSS para Tailwind
autoprefixer	10.x	Prefijos CSS para compatibilidad de navegadores

5. Archivos Creados y Modificados

5.1 Archivos nuevos creados

Archivo	Descripción
src/lib/firebase.ts	Inicialización de Firebase: initializeApp, getAuth, getFirestore con variables de entorno
src/lib/auth.ts	Funciones de autenticación: login, register (con validación de nombre único), logout
src/lib/firestore.ts	Todas las operaciones Firestore: loadUserData, saveUserData, saveBattle, loadBattles, loadRanking, loadSword, loadSparkles
src/lib/types/index.ts	Definiciones TypeScript: User, BattleRecord, Equipment, ShopItem, RankedHero, ChatMessage, NavSection
src/lib/data/mock.ts	Datos mock para desarrollo: MOCK_USER, MOCK_VITALITY, MOCK_EQUIPMENT_SET, MOCK_SHOP_ITEMS
src/lib/utils/index.ts	Utilidades: cn() para combinación condicional de clases Tailwind con clsx/twMerge
src/app/login/page.tsx	Pantalla de login con formulario email/password y validación
src/app/register/page.tsx	Pantalla de registro con nombre de héroe único, email y contraseña
src/app/api/oracle/route.ts	API Route Next.js: proxy seguro hacia Cloud Function con ORACLE_FUNCTION_URL

src/components/layout/Sidebar.tsx	Sidebar de navegación: logo, nav items, logout, saldo de oro, indicador online
src/components/layout/TopBar.tsx	Barra superior: búsqueda, stats HP/XP, notificaciones, perfil de usuario
src/components/dashboard/WelcomeBanner.tsx	Banner de bienvenida con nombre real, nivel, combates semanales
src/components/dashboard/VitalityStats.tsx	Gráfica Recharts de vitalidad semanal con panel de entrada de datos
src/components/oracle/GeminiOracle.tsx	Chat en tiempo real con Gemini AI: mensajes, loading state, scroll automático
src/components/ranking/HeroesRanking.tsx	Tabla de ranking global con avatares emoji y niveles
src/components/inventory/EquipmentInventory.tsx	Grid de inventario con sistema de rarezas y sets de equipo
src/components/battles/BattlesHistory.tsx	Historial filtrable de combates por tipo de hábito y fecha
src/components/shop/GoldShop.tsx	Tienda con catálogo de Firestore, filtros por categoría, compra con oro
src/components/profile/ProfileSection.tsx	Sección de perfil editable del usuario
src/components/settings/SettingsSection.tsx	Panel de configuración de preferencias
src/components/guide/GuideSection.tsx	Guía de usuario con secciones de mecánicas
.env.local	Variables de entorno: Firebase config + ORACLE_FUNCTION_URL (no subido al repo)

5.2 Archivos modificados

Archivo	Cambio realizado
package.json	Agregadas dependencias: firebase 10.14.1, recharts 2.12.7
src/app/page.tsx	Refactorizado completamente: verificación de sesión con onAuthStateChanged, carga paralela de datos reales de Firestore
tailwind.config.ts	Agregados tokens de diseño personalizados: mq-bg, mq-card, mq-blue, mq-gold, mq-text, mq-muted, etc.
src/app/globals.css	Variables CSS globales para el tema dark de MindQuest
src/app/layout.tsx	Metadata del proyecto, fuente global, viewport

6. Variables de Entorno

Archivo **.env.local** en la raíz del proyecto. Nunca se sube al repositorio (.gitignore).

Variable	Acceso	Descripción
NEXT_PUBLIC_FIREBASE_API_KEY	Cliente	API key del proyecto Firebase (pública por diseño)
NEXT_PUBLIC_FIREBASE_AUTH_DOMAIN	Cliente	Dominio de autenticación Firebase
NEXT_PUBLIC_FIREBASE_PROJECT_ID	Cliente	ID del proyecto Firebase
NEXT_PUBLIC_FIREBASE_STORAGE_BUCKET	Cliente	Bucket de Storage Firebase
NEXT_PUBLIC_FIREBASE_MESSAGING_SENDER_ID	Cliente	Sender ID para notificaciones push
NEXT_PUBLIC_FIREBASE_APP_ID	Cliente	App ID de la app web en Firebase Console
ORACLE_FUNCTION_URL	Servidor	URL de la Cloud Function — NUNCA llega al navegador

El prefijo **NEXT_PUBLIC_** hace que la variable sea embebida en el bundle del cliente. **ORACLE_FUNCTION_URL** no lo lleva — solo existe en el proceso del servidor Next.js y el navegador nunca puede acceder a ella.

7. Estructura de Firestore

Idéntica a la de la app Android. Ambas plataformas leen y escriben los mismos documentos.

Colección / Documento	Campo	Tipo	Descripción
users/{uid}	displayName	String	Nombre único de héroe
users/{uid}	email	String	Correo del usuario
users/{uid}	heroLevel	Number	Nivel actual del héroe
users/{uid}	heroXp	Number	XP en nivel actual (0-99)
users/{uid}	heroGold	Number	Oro disponible
users/{uid}	heroHp	Number	HP actual
users/{uid}	heroMaxHP	Number	HP máximo (base + equipo)
users/{uid}	totalXp	Number	XP total histórico (ranking)
users/{uid}	heroClass	String	Clase de héroe (Web-only por ahora)
users/{uid}/battles/{id}	date	Timestamp	Fecha del combate
users/{uid}/battles/{id}	habitType	String	Tipo de hábito del monstruo
users/{uid}/battles/{id}	result	String	"Victoria" o "Derrota"
users/{uid}/battles/{id}	goldEarned	Number	Oro ganado
users/{uid}/battles/{id}	xpEarned	Number	XP ganado
users/{uid}/inventory/{id}	id	String	ID del ítem del catálogo
users/{uid}/inventory/{id}	name / stat / emoji	String	Metadatos del ítem
users/{uid}/inventory/{id}	price	Number	Precio pagado
users/{uid}/inventory/{id}	purchasedAt	Timestamp	Fecha de compra
users/{uid}/vitality/{date}	date	String	Fecha ISO (yyyy-mm-dd)
users/{uid}/vitality/{date}	weightLossKg	Number	Pérdida de peso del día
users/{uid}/vitality/{date}	activityMinutes	Number	Minutos de actividad del día
shop_catalog/{id}	name / emoji / stat	String	Datos del ítem en tienda
shop_catalog/{id}	price	Number	Precio en oro
shop_catalog/{id}	bonusHp / bonusPower	Number	Estadísticas del ítem
shop_catalog/{id}	category / rarity	String	Categoría y rareza (Web)
shop_catalog/{id}	iconName	String	Nombre de icono Lucide (Web)

8. Flujos Funcionales Detallados

8.1 Flujo de Autenticación

1. Usuario navega a / → page.tsx monta y ejecuta auth.onAuthStateChanged()
2. Sin sesión activa: callback recibe null → router.push('/login')
3. LoginPage: formulario email/password → auth.ts: login() → signInWithEmailAndPassword()
4. Login exitoso: onAuthStateChanged dispara con usuario → carga de datos
5. Sin cuenta: enlace a /register → RegisterPage

6. RegisterPage: auth.ts: register() ejecuta:
 - a. isDisplayNameTaken() — consulta Firestore para unicidad del nombre de héroe
 - b. createUserWithEmailAndPassword() — crea cuenta en Firebase Auth
 - c. updateProfile() — guarda displayName en perfil de Firebase Auth
 - d. setDoc(doc(db, 'users', uid)) — crea documento con stats iniciales en Firestore
7. Logout: Sidebar → logout() → signOut(auth) → router.push('/login')

8.2 Flujo de Carga de Datos del Dashboard

1. onAuthStateChanged detecta sesión activa con firebaseUser
2. Promise.all paralelo para eficiencia máxima:
 - a. loadUserData() → users/{uid} completo
 - b. loadBattles() → users/{uid}/battles/ (ordenado por fecha desc, limit 50)
 - c. loadRanking() → collection('users') orderBy('totalXp', desc) limit(10)
 - d. loadInventory() → users/{uid}/inventory/ (solo IDs)
 - e. loadShopCatalog() → collection('shop_catalog') completo
3. computeStatBonuses(inventoryIds) → calcula bonusHp del equipo
4. setUser() con datos reales: heroHp y heroMaxHp incluyen bonusHp del equipo
5. setBattles() mapeando Timestamps a strings ISO para las tarjetas
6. setRanking() con índice de posición (i+1 = rank)
7. refreshVitality() → loadWeeklyVitality() para los últimos 7 días de VitalityStats
8. isLoadingUser = false → dashboard renderiza con datos reales

8.3 Flujo del Oráculo Gemini (Web)

1. Usuario escribe en GeminiOracle.tsx → presiona Send o Enter
2. handleSend(): agrega ChatMessage con role='user' al estado local
3. isLoading = true → mensaje placeholder 'El Oráculo consulta el cosmos...'
4. fetch('/api/oracle', {method:'POST', body: JSON.stringify({message})})
5. route.ts (servidor) recibe el POST desde el browser
6. route.ts hace fetch a ORACLE_FUNCTION_URL con el mismo body
7. Cloud Function llama a Gemini API (gemini-2.5-flash) con prompt RPG
8. Respuesta epica → route.ts → browser → ChatMessage role='oracle' agregado
9. useEffect en scrollRef hace auto-scroll al último mensaje

8.4 Flujo de Vitalidad Semanal

VitalityStats es un componente exclusivo de la plataforma web que permite registrar métricas diarias:

1. Al cargar el dashboard: loadWeeklyVitality() itera los últimos 7 días
2. Para cada fecha lee users/{uid}/vitality/{yyyy-mm-dd} (setDoc con merge)
3. Si el documento no existe, devuelve valores en cero para ese día
4. Recharts renderiza BarChart con los 7 días y sus métricas
5. El usuario puede registrar una nueva entrada diaria → saveVitalityEntry()
6. onStatsUpdated callback → refreshVitality() actualiza la gráfica

8.5 Flujo de Compra en la Tienda

1. GoldShop recibe shopItems (del catálogo de Firestore) y purchasedIds
2. Filtrado visual: ítems ya comprados marcados / deshabilitados
3. Usuario presiona comprar → handlePurchase(item) en page.tsx
4. Validación local: user.gold >= item.price
5. spendGold(amount, item) en firestore.ts — operación Firestore:
 - a. Lee heroGold actual del documento del usuario
 - b. updateDoc: heroGold = current - amount
 - c. addDoc en inventory/ con metadatos del ítem y serverTimestamp()
6. Estado local actualizado: gold reducido, heroHp/heroMaxHp con bonusHp del ítem
7. purchasedIds actualizado → EquipmentInventory se re-renderiza con el nuevo ítem

9. Componentes y Páginas — Detalles de Implementación

9.1 Arquitectura de Componentes

La estructura de componentes sigue el patrón de **composición y prop-drilling controlado**. El estado global vive en page.tsx (DashboardPage) y se pasa hacia abajo por props. Los componentes son funcionales puros con TypeScript estricto.

Componente	Props principales	Conexión con Firestore
page.tsx (DashboardPage)	—	Sí — carga y gestiona todo el estado
Sidebar	activeSection, onNavigate, userGold	No — recibe datos
TopBar	user: User	No — recibe datos
WelcomeBanner	user, weeklyBattles	No — recibe datos
VitalityStats	stats, onStatsUpdated	Sí — saveVitalityEntry, loadWeeklyVitality
GeminiOracle	initialMessages	Indirecto — vía /api/oracle
HeroesRanking	heroes: RankedHero[]	No — recibe datos
EquipmentInventory	equipment, activeSet, totalSlots	No — recibe datos filtrados
BattlesHistory	battles: BattleRecord[]	No — recibe datos
GoldShop	items, userGold, onPurchase, purchasedIds	Indirecto — vía handlePurchase
ProfileSection	user, onUserUpdated	Puede escribir en Firestore
SettingsSection	—	No
GuideSection	—	No

9.2 Sistema de Tipos TypeScript (src/lib/types/index.ts)

El proyecto usa TypeScript estricto con interfaces bien definidas para toda la capa de datos:

Tipo / Interface	Descripción
User	id, username, displayName, heroClass, level, xp, xpToNextLevel, totalXp, gold, heroHp, heroMaxHp, avatarUrl,
HeroClass	Union type: Guerrero Sanador Asesino Mago Caballero Arquero Druida Paladín
BattleRecord	id, date (ISO string), habitType, result, goldEarned, xpEarned, userId

HabitType	Union type: 'Agua' 'Postura' 'Mente'
BattleResult	Union type: 'Victoria' 'Derrota'
Equipment	id, name, type (EquipmentType), rarity (EquipmentRarity), stats, locked, iconName
EquipmentStats	power?, defense?, health? (todos opcionales Number)
EquipmentRarity	Union type: 'common' 'rare' 'epic' 'legendary'
ShopItem	id, name, description, price, category, rarity, stats, iconName, bonusHp, bonusPower
RankedHero	rank, userId, name, level, gold, heroClass, avatarEmoji
ChatMessage	id, role (ChatRole), text, timestamp (ISO string)
ChatRole	Union type: 'oracle' 'user'
NavSection	Union type: 'dashboard' 'battles' 'inventory' 'oracle' 'ranking' 'profile' 'settings' 'guide'
WeeklyVitalityStats	entries: DailyVitalityEntry[], totalWeightLossKg, avgActivityMinutes
DailyVitalityEntry	date (ISO string), weightLossKg, activityMinutes
EquipmentSet	id, name, bonus, equipmentIds: string[]

9.3 Secciones del Dashboard (renderSection)

page.tsx implementa un renderizador de secciones con switch statement. Cada sección renderiza los componentes relevantes para esa vista:

Sección (NavSection)	Componentes renderizados
dashboard	WelcomeBanner + VitalityStats + GeminiOracle + HeroesRanking + EquipmentInventory + BattlesHistory + GoldShop
battles	BattlesHistory
inventory	EquipmentInventory + GoldShop
oracle	GeminiOracle
ranking	HeroesRanking
profile	ProfileSection
settings	SettingsSection
guide	GuideSection

9.4 API Route — /api/oracle/route.ts

La API Route actúa como proxy seguro. Es la única forma de que el navegador pueda llamar a la Cloud Function sin exponer su URL:

- GET:** Método no soportado → 405
- POST sin body/message:** Validación → 400 Bad Request
- POST válido:** fetch a ORACLE_FUNCTION_URL → 200 con reply, o 502 si falla
- Error interno:** catch → 500 Internal Server Error

10. Sistema de Diseño (Tailwind Tokens)

El proyecto define tokens de diseño personalizados en tailwind.config.ts bajo el namespace 'mq-*':

Token Tailwind	Hex	Uso en la UI
mq-bg	#0f172a	Fondo principal (sidebar, página)
mq-card	#1e293b	Cards, paneles, inputs
mq-card2	#253347	Cards secundarias, hover states
mq-blue	#3b82f6	Accent principal: nav activo, botones, links
mq-blue2	#1d4ed8	Gradientes, sombras de botones
mq-gold	#f59e0b	Oro del juego, oráculo, highlights
mq-gold2	#d97706	Gradientes del oro, botón enviar oracle
mq-text	#e2e8f0	Texto principal
mq-muted	#94a3b8	Texto secundario, placeholders, labels

10.1 Efectos Visuales del Dashboard

El layout principal incluye efectos de ambient glow para reforzar la estética sci-fi:

- Círculo azul semitransparente (blur-150px) en el cuadrante superior derecho
- Círculo dorado semitransparente (blur-150px) en el cuadrante inferior izquierdo
- Ambos son `pointer-events-none` y `fixed` para no interferir con la UI
- TopBar tiene un glow adicional centrado que refuerza el color azul del sistema

11. Configuración e Instalación

11.1 Requisitos

- Node.js 18+ (requerido por Next.js 14)
- npm o yarn como gestor de paquetes
- Cuenta de Firebase con proyecto MindGuardians configurado
- Cloud Function desplegada (compartida con Android — mismo proyecto Firebase)

11.2 Pasos de Instalación

```
npm install
```

1. Crear archivo `.env.local` en la raíz con las variables de Firebase y URL de Cloud Function

```
npm run dev
```

2. Abrir `http://localhost:3000` en el navegador

11.3 Build para Producción

```
npm run build
```

```
npm start
```

O desplegar en Vercel (plataforma recomendada para Next.js) con las mismas variables de entorno.

12. Seguridad

Aspecto de seguridad	Implementación
API key de Gemini	Firebase Secrets — nunca en código ni en navegador
URL de Cloud Function	Variable ORACLE_FUNCTION_URL sin NEXT_PUBLIC_ — solo servidor
.env.local	.gitignore — nunca subido al repositorio
Autenticación de sesión	Firebase Auth con onAuthStateChanged — redirige a /login si no hay sesión
Protección del dashboard	page.tsx redirige automáticamente si firebaseUser es null
Validación de nombre de héroe	isDisplayNameTaken() verifica Firestore antes de registrar
Compras en tienda	spendGold() verifica heroGold en Firestore antes de descontar
Firestore durante desarrollo	Modo Test (30 días) — reglas de producción en entrega final
CORS en la API Route	Next.js maneja CORS automáticamente al ser same-origin
Variables de entorno	NEXT_PUBLIC_* solo para config de Firebase (públicas por diseño de Firebase)

13. Sincronización con la App Android

Ambas plataformas comparten el proyecto Firebase **mindguardians-b07d3** y la misma colección users en Firestore:

- Un usuario registrado en Android puede iniciar sesión en la web con las mismas credenciales
- El progreso del héroe (level, gold, xp, hp) se refleja en ambas plataformas en tiempo real
- Las batallas guardadas desde Android aparecen en el historial web (BattlesHistory)
- El ranking global es compartido — ordenado por totalXp desde ambas plataformas
- El catálogo de la tienda (shop_catalog/) es leído por ambas plataformas
- Los ítems comprados en cualquier plataforma aparecen en el inventario de la otra
- La Cloud Function de Python es compartida — mismo endpoint para Android y Web

13.1 Diferencias entre plataformas

Aunque comparten base de datos, cada plataforma tiene características únicas:

Característica	Android	Web
Combate RPG completo	Sí — pantalla dedicada	No (estadísticas solo)
Chat Oráculo	Sí — OracleScreen	Sí — GeminiOracle.tsx
Narración de ataques	Sí — Gemini en tiempo real	No
Bosses dinámicos	Sí — por debilidades reportadas	No
Vitalidad semanal (Recharts)	No	Sí — gráfica interactiva
Perfil editable	No	Sí — ProfileSection
Ajustes	No	Sí — SettingsSection
Guía de usuario	Sí — GuideScreen	Sí — GuideSection
Inventario visual	Sí — InventoryScreen	Sí — EquipmentInventory
Tienda	Sí — catálogo de Firestore	Sí — catálogo de Firestore
Ranking	Sí — RankingScreen	Sí — HeroesRanking

14. Pantallas, Rutas y Componentes del Dashboard

14.1 Rutas de la Aplicación

Ruta	Archivo	Descripción
/login	app/login/page.tsx	Inicio de sesión con email y contraseña
/register	app/register/page.tsx	Registro con nombre de héroe único, email y contraseña
/	app/page.tsx	Dashboard principal — protegido por Auth, renderiza sección activa
/api/oracle	app/api/oracle/route.ts	API Route POST: proxy seguro hacia Cloud Function de Gemini

14.2 Componentes por Sección del Dashboard

Componente	Sección	Descripción
WelcomeBanner	dashboard	Saludo con nombre real, nivel, racha de combates semanales
VitalityStats	dashboard	Gráfica Recharts de vitalidad (peso, actividad) los últimos 7 días con panel de entrada
GeminiOracle	dashboard / oracle	Chat en tiempo real con Gemini AI vía /api/oracle
HeroesRanking	dashboard / ranking	Top 10 héroes por totalXp con avatares emoji
EquipmentInventory	dashboard / inventory	Grid de equipo comprado, sistema de rarezas (common/rare/epic/legendary)
BattlesHistory	dashboard / battles	Historial filtrable por tipo de hábito (Agua/Postura/Mente) y fecha
GoldShop	dashboard / inventory	Tienda con catálogo de Firestore, filtros por categoría, compra con oro real
ProfileSection	profile	Edición de perfil del héroe (nombre, clase, etc.)
SettingsSection	settings	Panel de configuración de preferencias
GuideSection	guide	Guía interactiva de mecánicas del juego y hábitos